

Movie Ticket Booking System

Technical & Architectural Report

Course	Python Essentials
Reg. No.	26BOE10041
Name	Krishna Priya

Executive Summary

The Movie Ticket Booking System is a modular, lightweight Python application designed to handle ticket reservations, availability checks, cancellations, and billing calculations. It uses a JSON-backed persistence layer (`data/movies.json`) to store state without requiring any external database.

This document is a technical reference covering the system architecture, flowcharts, algorithms, pseudocode, data model, and verification strategy.

1. System Architecture

The application follows a **Modular Layered Architecture** that separates the User Interface (CLI), Business Logic Services, Data Access Layer, and Storage. Each layer communicates only with the layer directly beneath it, which keeps the modules easy to test and replace.

Architectural Blueprint

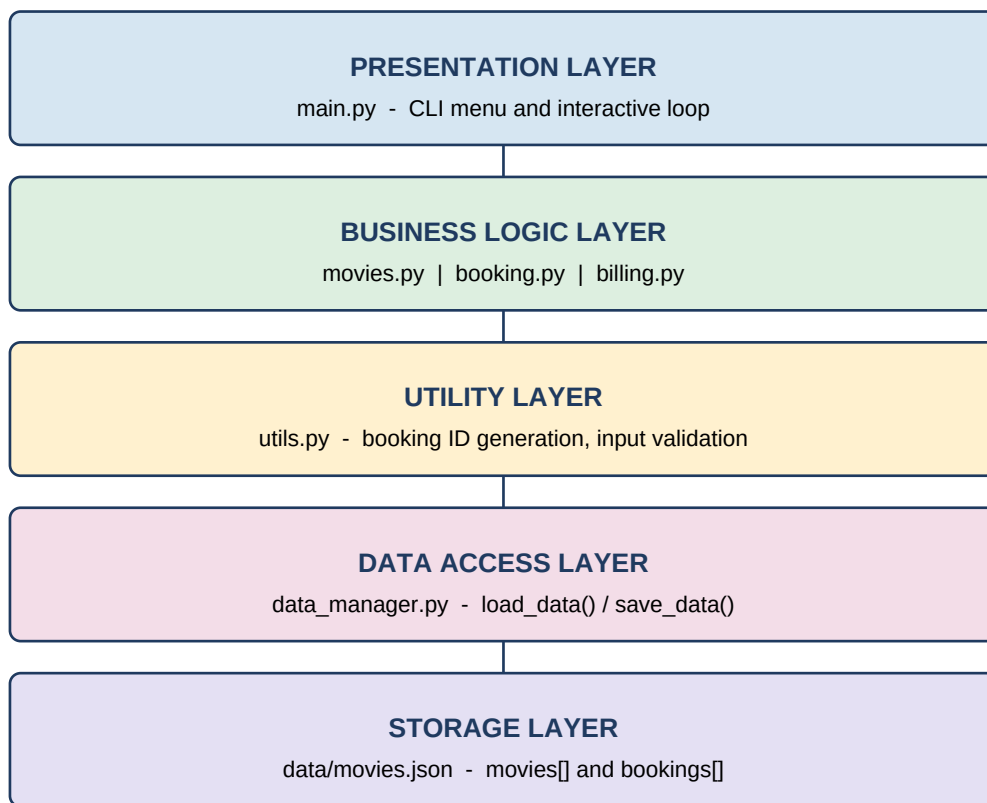


Figure 1: Layered architecture of the system

2. Module & File Breakdown

Module / File	Description	Key Functions / Elements
main.py	Main CLI driver and interactive loop	main()
movies.py	Movie catalog and seat viewing service	show_movies(), show_available_seats()
booking.py	Reservation and cancellation manager	find_movie(), book_seat(), cancel_booking(), show_bookings()
billing.py	Pricing and ticket calculation service	calculate_ticket_amount(), show_bill()
data_manager.py	Low-level JSON file I/O persistence	load_data(), save_data()
utils.py	Helper functions (validation and generation)	generate_booking_id(), get_positive_integer()
movies.json	Application data store	Movies, seats, booked seats, active bookings
test_project.py	Automated test suite	test_ticket_amount(), test_booking_and_cancellation()

3. Data Schema (movies.json)

The data file `movies.json` maintains two top-level collections: a **movies** array and a **bookings** array. Every booking references its movie through `movie_id`, and the booked seat is mirrored in the movie's `booked_seats` list.

```
{
  "movies": [
    {
      "id": "M1",
      "name": "Campus Days",
      "language": "Hindi",
      "price": 150,
      "seats": ["A1", "A2", "A3", "A4", "A5",
               "B1", "B2", "B3", "B4", "B5"],
      "booked_seats": ["A1"]
    }
  ],
  "bookings": [
    {
      "booking_id": "B3481",
      "movie_id": "M1",
      "movie_name": "Campus Days",
      "seat": "A1",
      "amount": 150
    }
  ]
}
```

4. Flowcharts & Process Workflows

4.1 Main Menu Execution Loop

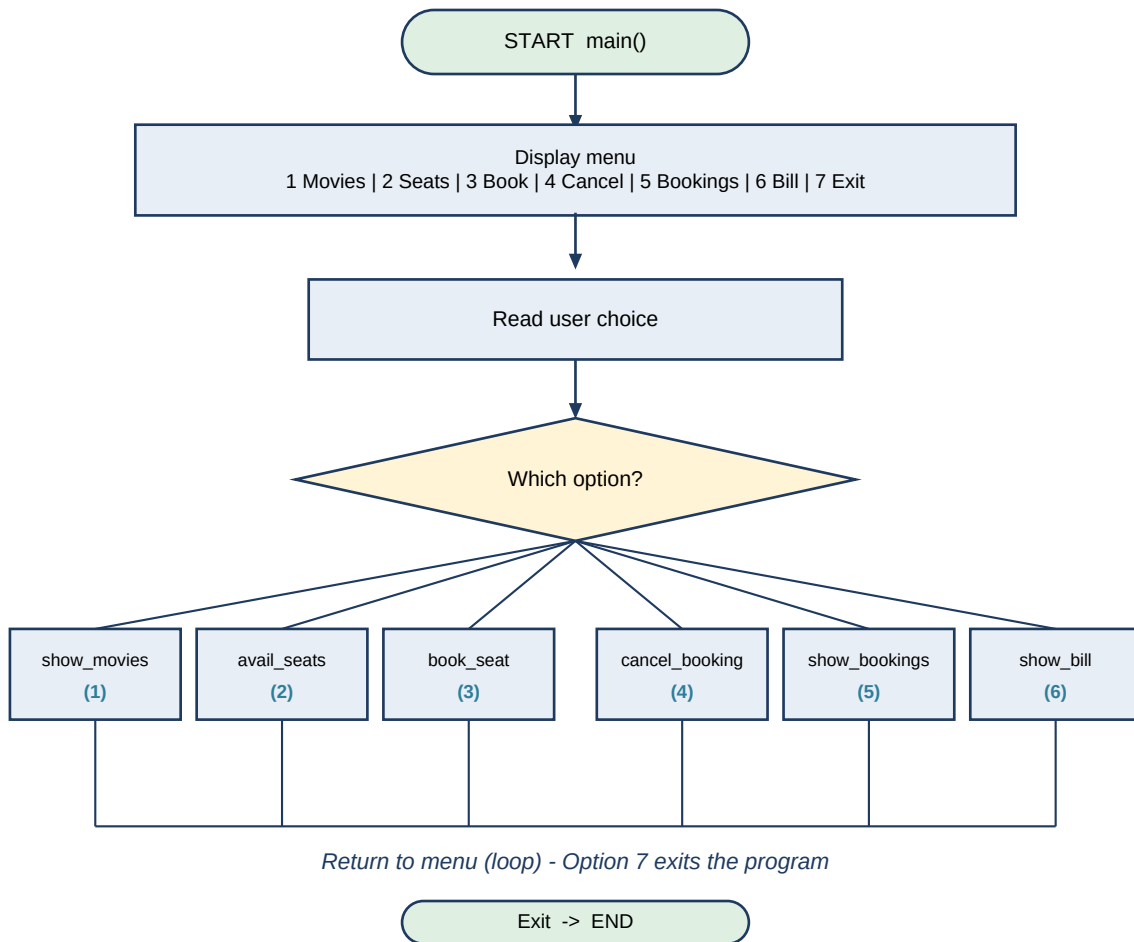


Figure 2: Main menu loop

The program repeatedly shows the menu, reads the user's choice, dispatches to the matching service function, and returns to the menu until the user selects Exit.

4.2 Seat Reservation Flowchart (book_seat)

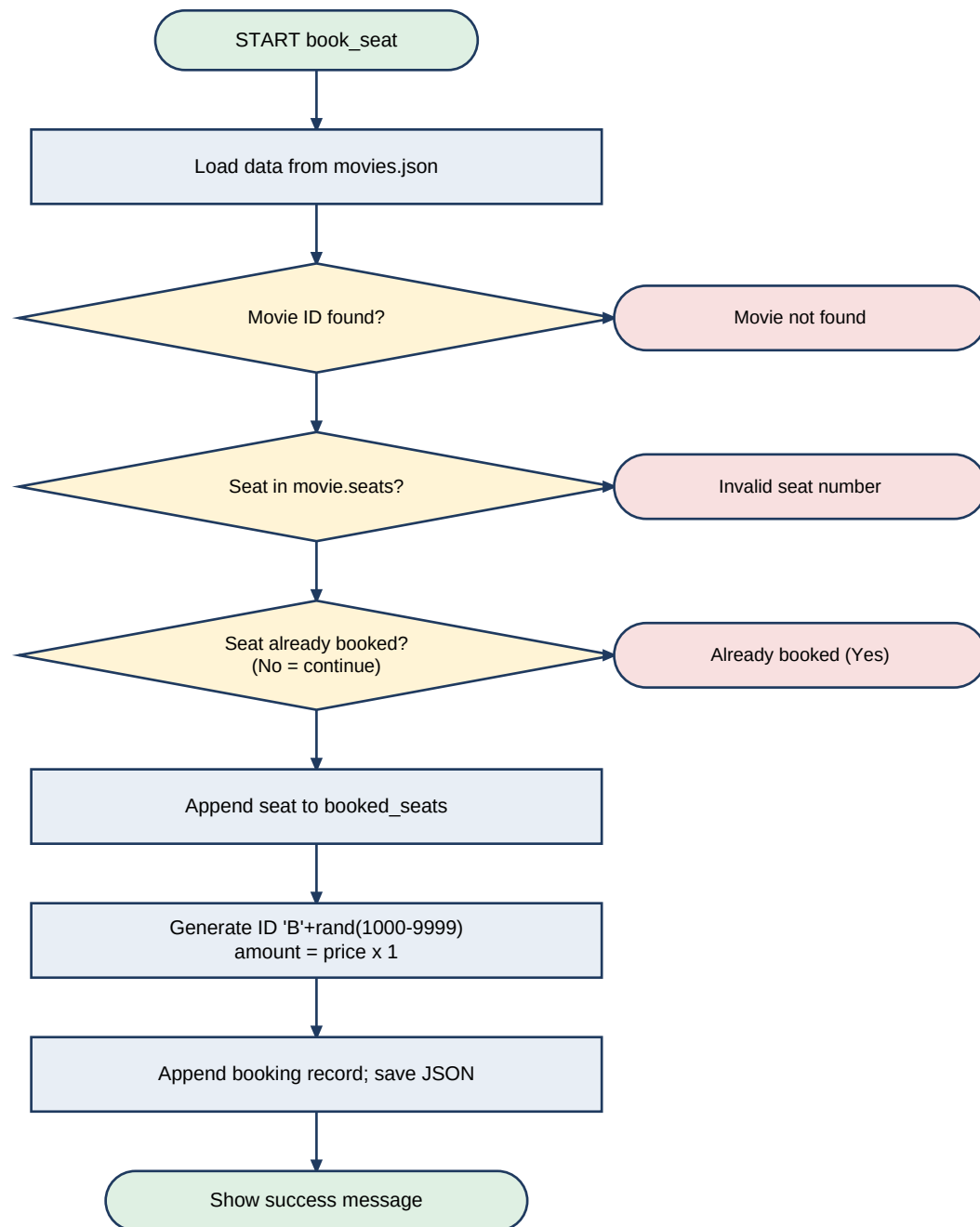


Figure 3: Seat reservation workflow

4.3 Booking Cancellation Flowchart (cancel_booking)

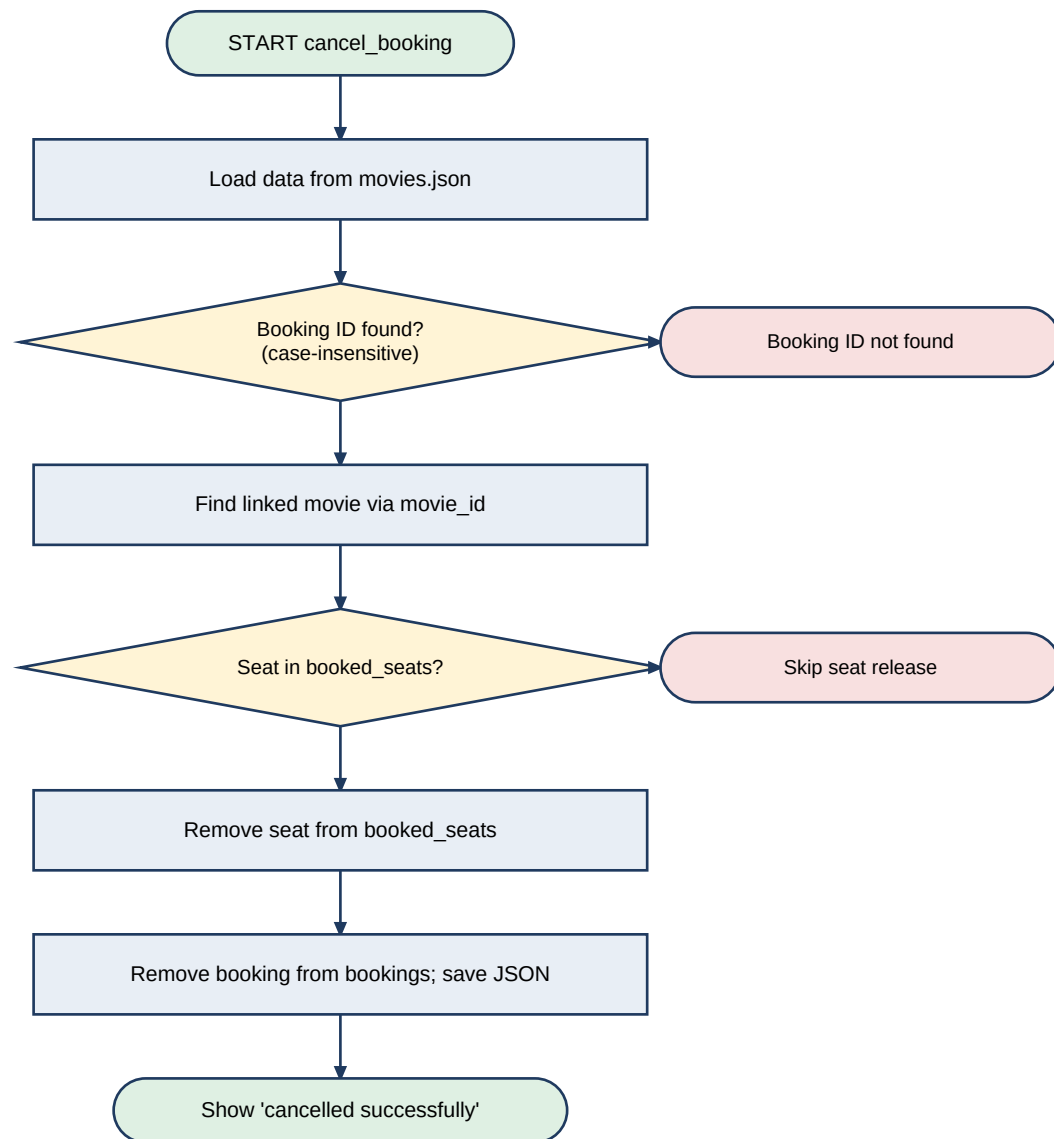


Figure 4: Booking cancellation workflow

5. Algorithms & Pseudocode

Algorithm 1: Seat Reservation (book_seat)

```
ALGORITHM BookSeat(movie_id, seat):
    INPUT:  movie_id (string), seat (string)
    OUTPUT: status_message (string)
    1. data <- LoadDataFromJSON()
    2. target_movie <- Null
    3. FOR EACH movie IN data.movies DO:
        IF LowerCase(movie.id) == LowerCase(movie_id) THEN:
            target_movie <- movie
            BREAK
    4. IF target_movie IS Null THEN:
        RETURN "Movie not found."
    5. IF seat NOT IN target_movie.seats THEN:
        RETURN "Invalid seat number."
    6. IF seat IN target_movie.booked_seats THEN:
        RETURN "This seat is already booked."
    7. APPEND seat TO target_movie.booked_seats
    8. booking_id <- "B" + RandomInteger(1000, 9999)
    9. amount <- target_movie.price * 1
    10. booking_record <- {
        "booking_id": booking_id,
        "movie_id":   target_movie.id,
        "movie_name": target_movie.name,
        "seat":       seat,
        "amount":     amount
    }
    11. APPEND booking_record TO data.bookings
    12. SaveDataToJSON(data)
    13. RETURN Formatted Success Message
END ALGORITHM
```

Algorithm 2: Booking Cancellation (cancel_booking)

```
ALGORITHM CancelBooking(booking_id):
    INPUT:  booking_id (string)
    OUTPUT: status_message (string)
    1. data <- LoadDataFromJSON()
    2. target_booking <- Null
    3. FOR EACH booking IN data.bookings DO:
        IF UpperCase(booking.booking_id) == UpperCase(booking_id) THEN:
            target_booking <- booking
            BREAK
    4. IF target_booking IS Null THEN:
        RETURN "Booking ID not found."
    5. target_movie <- FindMovie(data, target_booking.movie_id)
    6. IF target_movie IS NOT Null AND
        target_booking.seat IN target_movie.booked_seats THEN:
        REMOVE target_booking.seat FROM target_movie.booked_seats
    7. REMOVE target_booking FROM data.bookings
    8. SaveDataToJSON(data)
    9. RETURN "Booking cancelled successfully."
END ALGORITHM
```

Algorithm 3: Available Seats Computation (show_available_seats)

```
ALGORITHM ShowAvailableSeats():
  1. data <- LoadDataFromJSON()
  2. FOR EACH movie IN data.movies DO:
    available_list <- EMPTY_LIST
    FOR EACH seat IN movie.seats DO:
      IF seat NOT IN movie.booked_seats THEN:
        APPEND seat TO available_list
    PRINT movie.id + ". " + movie.name
    PRINT "Available: " + JoinStrings(available_list, ", ")
END ALGORITHM
```

6. Verification & Testing Framework

The system includes unit tests implemented in `test_project.py`.

Automated Tests Overview

test_ticket_amount() - Asserts the price calculation for movie M1 with quantity 2: $150 \times 2 = 300$.

test_booking_and_cancellation() - Resets the database state using `reset_data()`, books seat A1 for movie M1, and asserts that A1 appears in M1's `booked_seats`. It then cancels the booking using the generated `booking_id` and asserts that A1 has been removed from `booked_seats`.

7. Recommended System Enhancements

Area	Current State	Future Scope / Recommendation
Interface	Command Line Interface (CLI)	Build REST APIs (FastAPI/Flask) and a web frontend (React/HTML5)
Database	File-based JSON (movies.json)	Migrate to SQLite/PostgreSQL with an ORM (SQLAlchemy)
Concurrency	Single-user JSON writing	Implement row-level locking or DB transactions to avoid race conditions
Auth	No authentication	Add user roles (Customer vs. Admin) and auth tokens (JWT)

Conclusion. The Movie Ticket Booking System demonstrates clean separation of concerns, a simple and transparent storage model, and tested core operations. The recommended enhancements would allow it to grow from a single-user CLI tool into a multi-user, database-backed service.